DHBW Stuttgart

Karsten Keßler

Datenbanksysteme

5. Relationale Algebra

Kurs Informatik TINF25ITH
Stand: 15.10.2026

Inhalte der heutigen Vorlesung

- Übertragung des ER-Modells ins relationale Modell
- Datenzugriffe
- Relationale Algebra
- Operatoren
- Beispiel

Anfragesprachen

- Bisher wurde die Struktur der Daten beschrieben
  - Im ERM
  - Überführung in das relationale Modell
- Für die Abfrage der Daten braucht man eine Anfragesprache
  - Praktisch (immer) SQL
- Theoretisch gibt es drei Möglichkeiten
  - **Relationale Algebra**
  - Relationaler Domänenkalkül
  - Relationaler Tupelkalkül (Basis von SQL)

Relationale Algebra
Prozedural orientiert
DBMS nutzt dies grob für die Anfrageoptimierung

Relationenkalkül
Rein deklarativ
Bildet die Grundlage von SQL (dieses hat Erweiterungen)

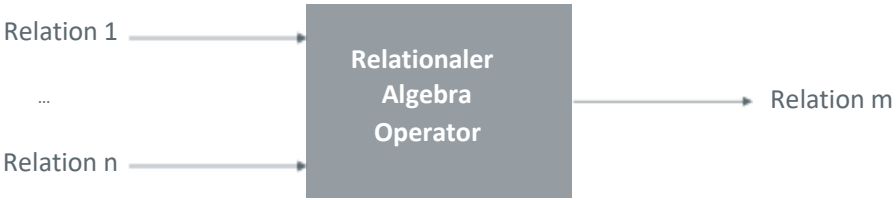
DHBW Stuttgart

Relationale Algebra

Warum Relationale Algebra?
- Grundlage aller SQL-Anfragen
- Theoretisches Fundament für Optimierung im DBMS
- Ermöglicht formale Beweise der Anfrageäquivalenz
- Verbindung zwischen Theorie (ERM) und Praxis (SQL)
- Erklärt, warum eine Anfrage schnell oder langsam ist

→ Formale Grundlage für relationale Datenbanksysteme. Besteht aus einer Menge von Operatoren, die auf Relationen angewendet werden, wie z. B. Selektion, Projektion, Vereinigungsmenge, Differenzmenge, Kreuzprodukt und Join.

- Alle Operatoren sind **mengenorientiert abgeschlossen**
- Jeder Operator bekommt als Eingabe eine (oder mehrere) Mengen von Tupeln und gibt eine Menge von Tupeln aus



SQL ist die praktische Implementierung der theoretischen Grundlagen der Relationalen Algebra und des Relationenkalküls. DBMS verwenden die Relationale Algebra im Hintergrund zur Abfrageoptimierung, während der Benutzer in SQL deklarative Abfragen formuliert.

DHBW Stuttgart

Basis-Operatoren in der Relationalen Algebra

Alle Beispiele zum Ausprobieren in RelaX

Name	Operator (Text)	Operator (mathematisch)
Selektion	SEL	σ_P (R) (Prädikat p, das erfüllt sein muss)
Projektion	PROJ	$\Pi_{a1,...an}$ (R) ($_{a1,...an}$ ausgewählte Attribute)
Kreuzprodukt	X	R x S
Vereinigung	UNION	R U S
Mengendifferenz	MINUS	R - S
Umbenennung	RENAME	ϱ_V (R) (neuer Name v)

1. Selektion: Auswahl der Zeilen

- SEL [SEMESTER = 5] (STUDENTEN)
- $\sigma_{\text{'Semester=5'}}$ (Studenten)

SEL [SEMESTER = 5 ∧ NAME = 'Müller'] (STUDENTEN)

$\sigma_{\text{'Semester=5 ∧ Name='Müller''}}$ (Studenten)

- Wählt alle Tupel der Relation STUDENTEN aus, welche die Bedingung (oder auch „das Prädikat“) SEMESTER = 5 erfüllen
- Operatoren
  - Logische Operatoren: ∧, ∨, ¬
  - Vergleichsoperatoren: =, ≥, ≤, >, >

```
SELECT * FROM Studenten
WHERE Semester = 5;

SELECT * FROM Studenten WHERE
Semester = 5 AND Name = 'Müller';
```

R

A	B	C
1	f	III
8	z	XI
5	a	V
2	q	VI
9	c	II
2	c	IX
1	f	X

$\sigma_{A=2}(R)$

A	B	C
2	q	VI
2	c	IX

$\sigma_{B\leq\text{'f'}}(R)$

A	B	C
1	f	III
5	a	V
9	c	II
2	c	IX
1	f	X

Beispiel nach: Rheinwerk Verlag, Bonn 2022

2. Projektion: Auswahl der Spalten

- PROJ [SEMESTER, NAME] (STUDENTEN)

$\Pi_{Semester,\ Name}$ (Studenten)

- Wählt aus der Relation (Tabelle) Studenten die Spalte Semester und Name aus.
- Alle anderen Attribute werden herausgefiltert.
- Was passiert mit Duplikaten?

  → Werden in der Relationalen Algebra automatisch herausgefiltert

SELECT Semester, Name FROM Studenten;

SELECT DISTINCT Semester, Name FROM Studenten

R

A	B	C
1	f	III
8	z	XI
5	a	V
2	q	VI
9	c	II
2	c	IX
1	f	X

$\pi_{A,B}(R)$

A	B
1	f
8	z
5	a
2	q
9	c
2	c

$\pi_{A}(R)$

A
1
8
5
2
9

$\pi_{B,C}(R)$

B	C
f	III
z	XI
a	V
q	VI
c	II
c	IX
f	X

Beispiel nach: Rheinwerk Verlag, Bonn 2022

3. Kreuzprodukt (= kartesisches Produkt)

- Bisher: Operatoren auf nur einer Relation => Verbindung von mehreren Relationen
  - R x S
- Beispielanfrage: “Gib alle Studenten und Professoren aus!“
- STUDENTEN x PROFESSOREN
  (oder PROFESSOREN x STUDENTEN)

SELECT * FROM Studenten CROSS JOIN Professoren
SELECT * FROM Studenten, Professoren

R × S × T × U
SELECT * FROM R CROSS JOIN S CROSS JOIN T CROSS JOIN U;
SELECT * FROM R, S, T, U;

Ein kartesisches Produkt erzeugt eine große Anzahl an Ergebnissen, da jede Zeile aus der ersten Tabelle mit jeder Zeile aus der zweiten Tabelle kombiniert wird. Daher kann die Anzahl der Zeilen in der resultierenden Tabelle erheblich sein (Produkt der Zeilenanzahlen beider Tabellen, hier: 2 x 4 = 8 Zeilen).

R

A	B	C
1	f	III
8	z	XI

S

D	E
7	e
2	i
9	f
4	o

R × S

A	B	C	D	E
1	f	III	7	e
1	f	III	2	i
1	f	III	9	f
1	f	III	4	o
8	z	XI	7	e
8	z	XI	2	i
8	z	XI	9	f
8	z	XI	4	o

Beispiel nach: Rheinwerk Verlag, Bonn 2022

In RelaX: **RK × S**

4. Vereinigung (Union): Relationen mit gleichem Schema zu einer Relation zusammenfassen

- Fasst zwei Relationen mit gleichem Schema (Attributname und Attributtyp) zu einer Relation zusammen. Die Namen der jeweiligen Spalten der beiden Tabellen müssen nicht identisch sein.
- ASSISTENTEN ∪ DOKTORANDEN

  SELECT PersNr, Name FROM Assistenten UNION SELECT PersNr, Name FROM Doktoranden

R

A	B	C
1	f	III
8	z	XI
3	o	XII
1	f	X

S

A	B	C
7	e	V
2	i	V
3	o	XII

R ∪ S

A	B	C
1	f	III
8	z	XI
3	o	XII
1	f	X
7	e	V
2	i	V

UNION entfernt standardmäßig alle Duplikate.
UNION ALL behält alle Tupel

Beispiel nach: Rheinwerk Verlag, Bonn 2022

In RelaX: **RU ∪ SU**

5. Mengendifferenz

- ASSISTENTEN - DOKTORANDEN
- Für zwei Relationen R und S mit gleichem Schema ist es die Menge der Tupel, die in R aber nicht in S sind. Duplikate werden standardmäßig entfernt

SELECT * FROM Assistenten WHERE Name NOT IN (SELECT Name FROM Doktoranden)

SELECT * FROM Assistenten MINUS SELECT * FROM Doktoranden

R

A	B	C
1	f	III
8	z	XI
3	d	V
3	o	XII
1	f	X

S

A	B	C
1	f	X
3	d	V

R − S

A	B	C
1	f	III
8	z	XI
3	o	XII

Beispiel nach: Rheinwerk Verlag, Bonn 2022

In RelaX: **RD − SD**

NOT IN: funktioniert überall, kann aber ineffizient bei großen Tabellen sein.

MINUS: Funktioniert in ORACLE.

EXCEPT: funktioniert in vielen Dialekten (PostgreSQL, SQLServer)

6. Umbenennung

$\varrho_{\text{NeuName} \leftarrow \text{Name}}$ (Doktoranden)

- Falls es z.B. notwendig ist eine Relation mehrfach in einer Anfrage zu verwenden. Dabei wird eine Kopie erzeugt, die einen neuen Namen braucht.
  Temporäre Umbenennung:
  SELECT Studentenname AS sn FROM Studenten;

Dauerhafte Änderung:
ALTER TABLE Doktoranden CHANGE Thema NeuThema varchar(50);

PostgreSQL, SQL Server, Oracle: Verwenden ALTER TABLE ... RENAME COLUMN ....
MySQL: ab Version 8.0 ebenfalls RENAME COLUMN; bei ALTER TABLE ... CHANGE ... muss der Datentyp angegeben werden.

Erweiterte Operatoren: Joins

Name	Operator (Text)	Operator (math.)	Was liefert der Operator?	Mit Basis-Operatoren
Theta-Join allgemeiner Inner Join	JOIN	$R \bowtie_{\theta} S$	Alle Tupelpaare aus R und S, die θ erfüllen (z. B. <, =, ≥).	$\sigma_{\theta}(R \times S)$ Selektion auf das kartesische Produkt
Equi-Join Spezialfall des Theta-Joins	EQUI JOIN	$R \bowtie_{R.A = S.B} S$	Theta-Join nur mit „=“; beide Vergleichsspalten bleiben erhalten.	$\sigma_{R.A = S.B}(R \times S)$ Selektion mit Gleichheitsbedingung
Natural Join Spezialfall des Equi-Joins	NATURAL JOIN	$R \bowtie S$	Equi-Join über alle gleichnamigen Attribute; gemeinsame Spalten nur einmal.	$\pi_{R \cup S}(\sigma_{R.A = S.A}(R \times S))$ Gleichheit auf allen gemeinsamen Attributen A, dann Projektion
Left Outer Join	LOJOIN	$R \mathbin{⟕} S$	Alle Tupel aus R; ohne Partner in S mit NULL ergänzt.	$(R \bowtie S) \cup ((R - \pi_R(R \bowtie S)) \times \omega_S)$ ω_S = {(NULL, …, NULL)} für die Attribute, die nur in S vorkommen
Right Outer Join	ROJOIN	$R \mathbin{⟖} S$	Alle Tupel aus S; ohne Partner in R mit NULL ergänzt.	$(R \bowtie S) \cup (\omega_R \times (S - \pi_S(R \bowtie S)))$ ω_R = {(NULL, …, NULL)} für die Attribute, die nur in R vorkommen
Full Outer Join	FOJOIN	$R \mathbin{⟗} S$	Alle Tupel aus R und S; fehlende Partner mit NULL ergänzt.	$(R \mathbin{⟕} S) \cup (R \mathbin{⟖} S)$ Vereinigung von Left und Right Outer Join

π_R, π_S: Projektion auf die Attribute von R bzw. S.
Textnotation wie SEL [ ] und PROJ [ ] auf Folie 6 und 7. Details mit Beispielen: Folie 14 bis 18.

Erweiterte Operatoren: Durchschnitt und Division

Name	Operator (Text)	Operator (math.)	Was liefert der Operator?	Mit Basis-Operatoren
Durchschnitt	INTERSECTION	$R \cap S$	Alle Tupel, die sowohl in R als auch in S vorkommen. R und S brauchen dasselbe Schema.	$R - (R - S)$ zweifache Differenz
Division Fragen mit „alle“	DIV	$R \div S$	Alle Werte der Attribute A, die in R mit jedem Tupel aus S kombiniert vorkommen.	$\pi_A(R) - \pi_A((\pi_A(R) \times S) - R)$ Projektion, kartesisches Produkt, Differenz

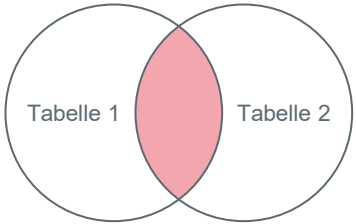
A: die Attribute von R, die nicht in S vorkommen. Voraussetzung für $R \div S$: Alle Attribute von S kommen auch in R vor.
Für $R \cap S$ (wie für $\cup$ und $-$) müssen R und S gleich viele Attribute mit gleichen Typen haben.
Details mit Beispielen: Durchschnitt Folie 19, Division Folie 21 und 22.

1. INNER JOIN (gemeinsame Schnittmenge)

nur die überlappende Menge beider Tabelleninhalte wird zurückgegeben

SELECT * FROM Professoren p INNER JOIN Vorlesungen v ON p.PersNr = v.gelesenVon (Equi-Join)

SELECT p.Name, p.Raum FROM Professoren p INNER JOIN Vorlesungen v ON p.PersNr = v.gelesenVon (Equi-Join)



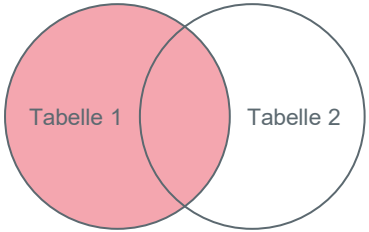
SELECT * FROM Professoren p INNER JOIN Vorlesungen v ON p.PersNr < v.gelesenVon; (Theta-Join)

SELECT * FROM Professoren p INNER JOIN Vorlesungen v ON p.PersNr > v.gelesenVon; (Theta-Join)

2. LEFT OUTER JOIN

die überlappende Menge beider Tabelleninhalte und alle Werte aus Tabelle 1

SELECT * FROM Professoren p LEFT [OUTER] JOIN Vorlesungen v ON p.PersNr = v.gelesenVon



SELECT * FROM Professoren p LEFT OUTER JOIN Vorlesungen v ON p.PersNr <> v.gelesenVon;

→Alle Datensätze der linken Tabelle (Professoren) werden zurückgegeben, unabhängig davon, ob sie eine Übereinstimmung in der rechten Tabelle (Vorlesungen) haben.

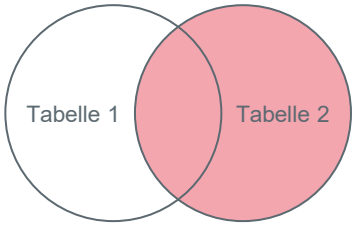
→ Die Bedingung für den Join ist ungleich (≠). Das bedeutet, dass die Datensätze in der rechten Tabelle (Vorlesungen) nur dann verbunden werden, wenn der Wert ungleich ist.

→ Wenn es keine passende Zeile in der rechten Tabelle gibt, wird der Wert aus der rechten Tabelle als NULL zurückgegeben (typisches Verhalten des Left Outer Joins).

3. RIGHT OUTER JOIN

die überlappende Menge beider Tabelleninhalte und alle Werte aus Tabelle 2

SELECT * FROM Professoren p RIGHT [OUTER] JOIN Vorlesungen v ON p.PersNr = v.gelesenVon

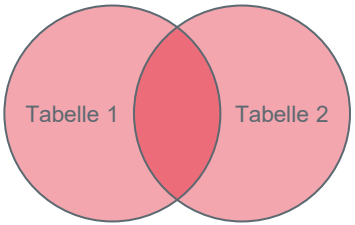


4. FULL OUTER JOIN

eine Kombination aus LEFT JOIN und RIGHT JOIN (in MySQL nicht direkt verfügbar)

SELECT * FROM Professoren p FULL [OUTER] JOIN Vorlesungen v ON p.PersNr = v.gelesenVon

SELECT * FROM Professoren p LEFT JOIN Vorlesungen v ON (p.PersNr = v.gelesenVon) UNION SELECT * FROM Professoren p RIGHT JOIN Vorlesungen v ON (p.PersNr = v.gelesenVon);



5. NATURAL JOIN

Allgemeine Form vom inneren Verbund. Tupel der beiden Tabellen werden verknüpft, wenn ihre Werte an den gleichnamigen Attributen übereinstimmen

SELECT * FROM Hochschulen NATURAL JOIN Senioren

R

A	B	C
1	f	3
8	z	1
5	a	2
2	q	3
9	c	2
2	c	1
1	f	1

S

C	D
1	abc
2	qrst
3	xyz

R ⋈ S

A	B	C	D
1	f	3	xyz
8	z	1	abc
5	a	2	qrst
2	q	3	xyz
9	c	2	qrst
2	c	1	abc
1	f	1	abc

Beispiel nach: Rheinwerk Verlag, Bonn 2022

In RelaX: **RJ ⋈ SJ**

6. INTERSECTION (Mengenoperation)

Liefert die Zeilen, die in beiden Tabellen (gleiches Schema) vorkommen.

SELECT * FROM Doktoranden WHERE Name IN (SELECT Name FROM Assistenten)

SELECT PersNr, Name FROM Assistenten INTERSECT SELECT PersNr, Name FROM Doktoranden

Beide Abfragen erzeugen eine Schnittmenge, der Operator INTERSECT ist jedoch genauer, wenn mehrere Spalten verglichen werden und sichergestellt werden soll, dass die Zeilen in allen Spalten übereinstimmen. Die WHERE IN-Klausel hingegen vergleicht nur eine spezifische Spalte (hier Name), während andere Spalten nicht berücksichtigt werden.

Mengenoperationen

- Voraussetzungen $R_1 \cap R_2; R_1 \cup R_2; R_1 - R_2$
  - Gleiche Anzahl von Attributen
  - Gleicher Typ
  - ⇒ Gleiches Schema

Prof1

PersNr	Name
1	Moerkotte
2	Kemper

Prof2

PersNr	Name
2	Kemper
3	Weikum

Schnittmenge: ***SELECT * FROM Prof1 INTERSECT SELECT * FROM Prof2;***

PersNr	Name
2	Kemper

Vereinigung: ***SELECT * FROM Prof1 UNION SELECT * FROM Prof2;***

PersNr	Name
1	Moerkotte
2	Kemper
3	Weikum

Differenz: ***SELECT * FROM Prof1 EXCEPT SELECT * FROM Prof2;***

PersNr	Name
1	Moerkotte

7. DIVISION

Fragen mit „alle“: Welche Piloten fliegen alle Flugzeuge aus R2 bzw. R3?

- $R_1 \div R_2$
- Voraussetzung: $\mathcal{R}_2 \subseteq \mathcal{R}_1$
- Werte, die in R_2 enthalten sind, müssen eine Teilmenge der Werte in R_1 sein

$$R_1 \div R_2 = \pi_{(\mathcal{R}_1 \setminus \mathcal{R}_2)}(R_1) \setminus \pi_{(\mathcal{R}_1 \setminus \mathcal{R}_2)}((\pi_{(\mathcal{R}_1 \setminus \mathcal{R}_2)}(R_1) \times R_2) \setminus R_1)$$

- Formale Definition:
  - $t \in R_1 \div R_2$, falls für jedes $v \in R_2$ ein $u \in R_1$ existiert, so dass gilt:
  - $u.\mathcal{R}_2 = v.\mathcal{R}_2$ und $u.(\mathcal{R}_1/\mathcal{R}_2) = t.(\mathcal{R}_1/\mathcal{R}_2)$
  - $(\mathcal{R}_1/\mathcal{R}_{2)}$ ist dabei die Menge der Attribute, die in R_1 enthalten ist, aber nicht in R_2

Vorgehen: R_1 auf PILOT projizieren und mit R_2 kreuzen; davon R_1 abziehen. Übrig bleiben die Piloten, denen ein Flugzeug aus R_2 fehlt. Diese werden von allen Piloten abgezogen; **das Ergebnis sind die Piloten, die alle Flugzeuge aus R_2 fliegen**. Für $R_1 \div R_3$ analog: R_3 enthält nur 707, die Snoopy, Meyer und Müller fliegen.

$R_1 \div R_2$
Snoopy
Müller

$R_1 \div R_3$
Snoopy
Meyer
Müller

Attribute, die in R_1 stehen, aber nicht in R_2.

R_1

PILOT	FLUGZEUG
Snoopy	707
Snoopy	727
Snoopy	747
Meyer	707
Meyer	727
Müller	707
Müller	727
Müller	747
Müller	777
Lüdenscheid	727

R_2

FLUGZEUG
707
727
747

R_3

FLUGZEUG
707

Weiteres Beispiel zur relationalen Division: Lieferant 3 liefert Artikel 500003 und 500004.
Gibt es weitere Lieferanten, die mindestens diese beiden Artikel liefern?

R: Lieferungen

LiefNr	ArtNr
5	500001
2	500002
1	500002
3	500003
4	500003
3	500004
4	500004
4	500005
1	500006
1	500007

S: Artikel von Lieferant 3

ArtNr
500003
500004

R ÷ S

LiefNr
3
4

Aufbau der Anfrage
$R = \pi_{LiefNr,\ ArtNr}(Lieferungen)$ $S = \pi_{ArtNr}(\sigma_{LiefNr\ =\ 3}(R))$
In einem Ausdruck: $Lieferungen \div \pi_{ArtNr}(\sigma_{LiefNr\ =\ 3}(Lieferungen))$

Herleitung mit Grundoperationen (Folie 13; alle Schritte in RelaX: Folie 23)
$A = \pi_{LiefNr}(R) = \{1, 2, 3, 4, 5\}$
$(A \times S) - R$: fehlende Kombinationen, 1, 2 und 5 mit je beiden Artikeln (6 Tupel)
$\pi_{LiefNr}((A \times S) - R) = \{1, 2, 5\}$

$R \div S = A - \{1, 2, 5\} = \{3, 4\}$
→ Lieferant 4 liefert ebenfalls beide Artikel; sein zusätzlicher Artikel 500005 stört nicht.

Lieferant 3 erfüllt die Bedingung trivial. Nur weitere Lieferanten: $\sigma_{LiefNr \neq 3}(R \div S) = \{4\}$.

```
In RelaX: Lieferungen ÷ π ArtNr (σ LiefNr = 3 (Lieferungen))
```

DHBW Stuttgart

Division Schritt für Schritt in RelaX

Schritt (Folie 22)	RelaX-Abfrage	Ergebnis
$S = \pi_{ArtNr}(\sigma_{LiefNr = 3}(R))$ Artikel von Lieferant 3	`π ArtNr (σ LiefNr = 3 (Lieferungen))`	500003, 500004
$A = \pi_{LiefNr}(R)$ alle Kandidaten	`π LiefNr (Lieferungen)`	1, 2, 3, 4, 5
$A \times S$ alle geforderten Kombinationen	`π LiefNr (Lieferungen) × π ArtNr (σ LiefNr = 3 (Lieferungen))`	10 Tupel
$(A \times S) - R$ fehlende Kombinationen	`(π LiefNr (Lieferungen) × π ArtNr (σ LiefNr = 3 (Lieferungen))) - Lieferungen`	6 Tupel: 1, 2 und 5 mit je beiden Artikeln
$\pi_{LiefNr}((A \times S) - R)$ Kandidaten mit Lücke	`π LiefNr ((π LiefNr (Lieferungen) × π ArtNr (σ LiefNr = 3 (Lieferungen))) - Lieferungen)`	1, 2, 5
$A - \pi_{LiefNr}((A \times S) - R)$ Ergebnis der Herleitung	`π LiefNr (Lieferungen) - π LiefNr ((π LiefNr (Lieferungen) × π ArtNr (σ LiefNr = 3 (Lieferungen))) - Lieferungen)`	3, 4
$R \div S$ Kontrolle mit dem Divisionsoperator	`Lieferungen ÷ π ArtNr (σ LiefNr = 3 (Lieferungen))`	3, 4
$\sigma_{LiefNr \neq 3}(R \div S)$ nur weitere Lieferanten	`σ LiefNr ≠ 3 (Lieferungen ÷ π ArtNr (σ LiefNr = 3 (Lieferungen)))`	4

Im RelaX-Datensatz heißt R „Lieferungen“; S entspricht der Relation „Artikel“. Jede Abfrage einzeln markieren und ausführen.

Gruppierung und Aggregation

Bei der Gruppierung werden Tupel mit gleichen Attributwerten gruppiert. Auf jede Gruppe wird eine Aggregatfunktion angewendet, die für die gesamte Gruppe einen einzelnen Wert berechnet. Z.B **SUM**, um die Werte eines spezifizierten Attributes zu summieren. COUNT, um die Anzahl der Elemente der Gruppe zu zählen. **MIN**, **MAX, AVG** um das Minimum, Maximum oder den Durchschnitt zu bestimmen.

Gruppierung

$\gamma_{A,f}$(R): Gruppiert die Relation R über das Attribut A und wendet auf die resultierenden Gruppen die Aggregatfunktion f an.

Sortierung

τ_{A}(R): Verwendet als Sortierkriterium für die Relation R das Attribut A.

```
SELECT Abteilung, SUM(Gehalt) FROM Mitarbeiter GROUP BY Abteilung;
SELECT * FROM Mitarbeiter ORDER BY Gehalt DESC;
```

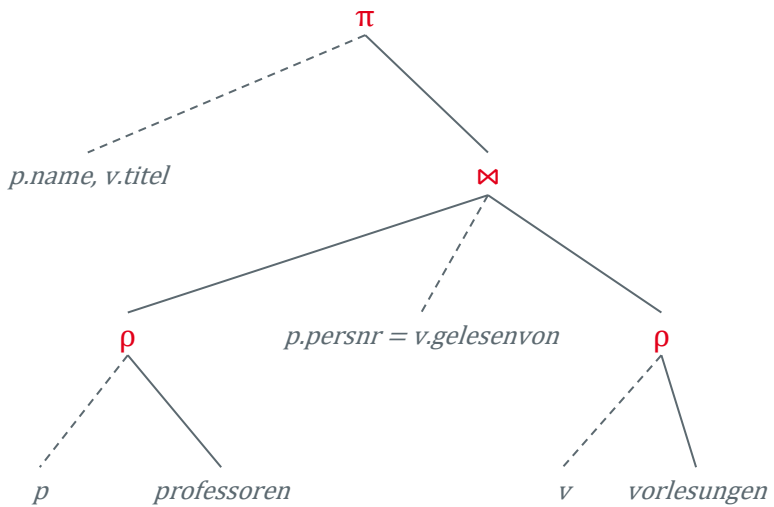
Abteilung	Name	Gehalt
IT	Anna	5000
IT	Ben	6000
Vertrieb	Clara	3500
Vertrieb	David	4000
HR	Eva	1500

$\gamma_{\text{Abteilung,SUM(Gehalt),COUNT(Name)}}$(Mitarbeiter)

Abteilung	Summe	Anzahl
IT	11000	2
Vertrieb	7500	2
HR	1500	1

Operatorbaumdarstellung der relationalen Algebra

$$\pi_{p.name,\ v.titel}\ (\rho_p\ professoren \bowtie_{p.persnr = v.gelesenvon} \rho_v\ vorlesungen)$$




```
SELECT p.Name, v.Titel
FROM Professoren p
INNER JOIN Vorlesungen v
ON p.PersNr = v.gelesenVon
```

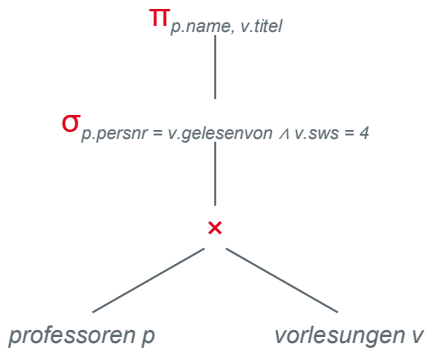
Name	Titel
Heinz	Funktionalanalysis
Heinz	Maßtheorie
Heinz	Wahrscheinlichkeitstheorie
Oswald	Partielle DGL

Join ⋈: Zuerst wird ein Theta-Join basierend auf der Bedingung p.persnr=v.gelesenvon durchgeführt, um die Tupel aus beiden Tabellen zu verknüpfen.
Projektion π: Nach dem Join wird eine Projektion auf die Spalten p.Name und v.Titel durchgeführt, um nur diese Spalten in der Ausgabe zu behalten.
Relationen ρ: Die Basisrelationen, nämlich die Tabellen Professoren p und Vorlesungen v, werden in den Blättern des Operatorbaums dargestellt.

(Baum mit Blättern und Knoten von unten nach oben lesen)

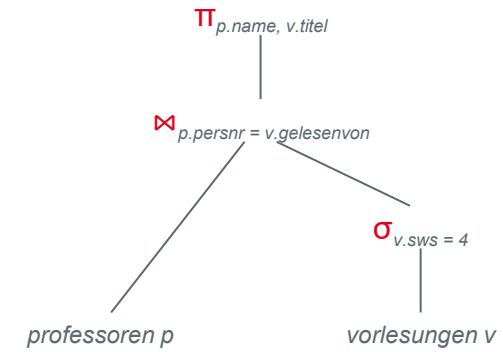
Optimierung: Selektionen früh ausführen

Naiver Plan



Kreuzprodukt aller Professoren mit allen Vorlesungen

Optimierter Plan



Nur Vorlesungen mit 4 SWS werden verbunden

- Beide Bäume liefern dasselbe Ergebnis; der rechte erzeugt kleinere Zwischenergebnisse.
- **Selektion aufteilen und nach unten schieben:** $\sigma_{F1 \wedge F2}(R) \equiv \sigma_{F1}(\sigma_{F2}(R))$; jede Bedingung wandert zu der Relation, die sie betrifft.
- **Kreuzprodukt und Selektion zum Join zusammenfassen:** $\sigma_{\theta}(R \times S) \equiv R \bowtie_{\theta} S$
- **Projektion früh ausführen:** Attribute, die weiter oben nicht gebraucht werden, fallen sofort weg.
- Der Anfrageoptimierer des DBMS wendet diese Äquivalenzregeln automatisch an.

Vom SQL-Befehl zum Ausführungsplan




```
EXPLAIN
SELECT p.Name, v.Titel
FROM Professoren p
INNER JOIN Vorlesungen v
  ON p.PersNr = v.gelesenVon
WHERE v.SWS = 4;
```

Ein Index auf v.SWS oder v.gelesenVon verändert den Plan: Statt die ganze Tabelle zu lesen, greift MySQL gezielt auf die passenden Zeilen zu.

- **EXPLAIN** zeigt den Plan, den MySQL gewählt hat, ohne die Anfrage auszuführen.
- **type:** Zugriffsart, z. B. ALL (ganze Tabelle lesen) oder ref (Zugriff über einen Index)
- **key:** der tatsächlich genutzte Index
- **rows:** geschätzte Zahl der Zeilen, die gelesen werden
- **EXPLAIN FORMAT=TREE** (ab MySQL 8.0.16) gibt den Plan als Operatorbaum aus.
- **EXPLAIN ANALYZE** (ab MySQL 8.0.18) führt die Anfrage aus und misst die Zeit je Operator.

Relationenkalkül und SQL

- **Relationale Algebra (RA)** ist relational vollständig, d.h. jeder Term kann in eine Anfragesprache (SQL) umgesetzt werden. Beschreibt wie Daten abgerufen werden sollen.
- **Relationaler Tupelkalkül (TRC):** Variablen werden an Tupel gebunden. Nützlich für korrekte Problemlösungen
- **Relationaler Domänenkalkül (DRC)** (Bereich)**:** ähnliche Idee, nur: Variablen werden an Wertemengen von Attributen gebunden. Nützlich für Projektionen/Teilergebnisse, Sicherheit, Sichten.

- Alle drei sind gleich mächtig
- **Relationenkalkül** (gilt für Tupelkalkül und Domänenkalkül)
  - beschreiben Ergebnistupel ohne Herleitungsvorschrift
  - mathematisches Prädikatenkalkül erster Stufe

  → deklarativ

  ∃: Existenzquantifizierung „es existiert ein…“
  ∀: Allquantifizierung „für alle…“

Bsp: Studenten s finden (**Tupelkalkül**, da Variable s Tupel aus Relationen binden)

$$\{s \mid s \in \mathit{Studenten} \wedge \exists h \in \mathit{hören}(s.\mathit{MatrNr} = h.\mathit{MatrNr} \wedge \exists v \in \mathit{Vorlesungen}(h.\mathit{VorlNr} = v.\mathit{VorlNr} \wedge \exists p \in \mathit{Professoren}(p.\mathit{PersNr} = v.\mathit{gelesenVon} \wedge p.\mathit{Name} = \text{'Pythagoras'})))\}$$

Bsp: MatrNr und Namen der Studenten finden (**Tupelkalkül**, da Tupel m,n verwendet werden, um Bedingungen für Abfrage zu bestimmen)

$$\{[m, n] \mid \exists s\,([m, n, s] \in \mathit{Studenten} \wedge \exists p, v, g\,([m, p, v, g] \in \mathit{prüfen} \wedge \exists a, r, b\,([p, a, r, b] \in \mathit{Professoren} \wedge a = \text{'Sokrates'})))\}$$

Weitere Beispielanfrage im relationalen **Tupelkalkül**:

{ p | p ∈ Professoren ∧ p.Rang = ‘Boss‘ } *SELECT * FROM Professoren WHERE Rang = 'Boss';*

→ Das Tupel p muss in der Relation Professoren enthalten sein
→ Das Tupel p muss für das Attribut Rang den Wert ‘Boss’ besitzen
→ Abfrage im Tupelkalkül, weil es mit ganzen Tupeln arbeitet.

Beispielanfrage im relationalen **Domänenkalkül**:

{ v, w, x, y, z | Personal(v, w, x, y, z) mit x = ‘Willi‘ } *SELECT * FROM Personal WHERE drittes_Attribut = 'Willi';*
oder:
{ v, w, Willi, y, z | Personal(v, w, ‘Willi‘, y z) }

falls nur der Nachname w erwünscht ist:
{w | ∃v, ∃y, ∃z Personal(v, w, ‘Willi‘, y z) }
→ Abfrage im Domänenkalkül, weil es mit einzelnen Attributen arbeitet.

Der relationale Domänenkalkül beschreibt, welche Attributwerte eine Abfrage erfüllen sollen, und diese Bedingungen lassen sich direkt in SQL übertragen, indem man die entsprechenden Spaltennamen und Werte in die SQL-Abfragestruktur einfügt. Domänenkalkül arbeitet mit Attributwerten und nicht mit ganzen Tupeln, daher kann man in SQL einfach die entsprechenden Spalten mit den Bedingungen abfragen.

Konzept	Beschreibung	Art	Beispiel
Relationale Algebra (RA)	Prozedural: beschreibt wie das Ergebnis berechnet wird (z. B. Join, Selektion, Projektion).	Imperativ (wie?, algorithmisch)	$\pi_{Name,Titel}$(Professoren $\bowtie_{PersNr=gelesenVon}$ Vorlesungen)
Relationaler Tupelkalkül (TRC)	Deklarativ: beschreibt welche Tupel im Ergebnis stehen; Variablen sind ganze Tupel.	Deklarativ (was? logisch, ergebnisorientiert)	{p | p ∈ Professoren ∧ p.Rang = 'Boss'}
Relationaler Domänenkalkül (DRC)	Deklarativ; Variablen sind Attributwerte (Domänen).	Deklarativ	{n | ∃r, d (Personal(r,n,d) ∧ d='Willi')}

SQL-Abbildung:

RA (Wie wird gerechnet?)	*SELECT p.Name, v.Titel FROM Professoren p JOIN Vorlesungen v ON p.PersNr = v.gelesenVon;*
TRC (Was soll im Ergebnis stehen?)	*SELECT * FROM Professoren WHERE Rang = 'Boss';*
DRC (Was soll auf Attribut-Ebene gelten?)	*SELECT n as Name FROM Personal WHERE d = 'Willi';*

DHBW Stuttgart

RA, TRC und SQL: gleiche Ergebnisse, unterschiedliche Rollen

Für klassische Anfragen drücken alle drei weitgehend dieselbe Ergebnismenge aus.

Sprache	Nutzen
Relationale Algebra (RA)	Beschreibt schrittweise, **welche Operatoren** auf Relationen angewendet werden, etwa Selektion, Projektion und Join. Wichtig, um Abfragepläne und Optimierungen zu verstehen.
Relationaler Tupelkalkül (TRC)	Beschreibt logisch, **welche Tupel** die gesuchte Eigenschaft erfüllen müssen. Deklarative, formale Grundlage für Anfragesprachen und zum Beweisen von Anfrageeigenschaften.
SQL	Die praktische Sprache realer Datenbanksysteme. Erweitert die theoretischen Grundlagen um Aggregationen, Sortierung, Änderungen an Daten und datenbankspezifische Funktionen.

Beispiel: Alle drei können „Namen der Studenten ab Semester 3“ ausdrücken. RA zeigt den Operatorbaum, TRC die logische Bedingung, und SQL ist die Form, die tatsächlich in MySQL, PostgreSQL oder SQLite ausgeführt wird.

Vielen Dank für Ihre Aufmerksamkeit!